

Labor: MQTT – Publish / Subscribe

C-Programmierung · 2 Sitzungen à 3 Stunden · Gruppen A + B

💡 *Warum diese Aufgabe? Im vorigen Labor habt ihr gelernt, Sensordaten aus einer Datei zu lesen und zu verarbeiten. Jetzt geht es einen Schritt weiter: Daten fließen nicht mehr aus einer Datei, sondern über ein Netzwerkprotokoll – in Echtzeit, asynchron, zwischen Programmen die nichts voneinander wissen müssen. Gleichzeitig übt ihr eine Kernkompetenz der professionellen Softwareentwicklung: eine Schnittstelle so präzise spezifizieren, dass zwei Teams unabhängig voneinander daran bauen können – und sich am Ende treffen.*

Wie dieses Labor funktioniert

Ihr arbeitet in Zweier-Gruppen: Gruppe A und Gruppe B. Beide Gruppen starten gemeinsam, trennen sich dann für die Implementierung und kommen zur Integration wieder zusammen. Das Labor läuft über zwei Sitzungen mit je 3 Stunden.

Gruppe A – implementiert:

- Sitzung 1: JSON-Serializer
- Sitzung 2: Verbindungslogik

Gruppe B – implementiert:

- Sitzung 1: Datengenerator
- Sitzung 2: Validator

Gemeinsam (alle 6): Konzept, Spec, Testvertrag, Integration, Reflexion

💡 *Wenn eine Gruppe ihre Aufgabe früher fertig stellt als die andere: Wählt die schwierigere der noch offenen Aufgaben und beginnt dort. Nicht warten – vorarbeiten.*

SITZUNG 1 · 3 Stunden

GEMEINSAM Phase 0 – MQTT verstehen (ca. 30 min)

💡 *Kein Code in dieser Phase. Erst verstehen, dann spezifizieren.*

0.1 – Das Grundprinzip

MQTT ist ein Nachrichtenprotokoll nach dem Publish/Subscribe-Prinzip. Schaut euch das folgende Diagramm an:



Beantwortet gemeinsam – schriftlich – die folgenden Fragen:

Wer ist in eurem Labor-Szenario der Publisher? Wer der Subscriber?

Was passiert, wenn der Subscriber abstürzt und neu startet – sind die Nachrichten weg?

Warum heißt das Protokoll nicht einfach "sende Daten an IP-Adresse"? Was ist der Vorteil des Topic-Konzepts?

0.2 – Topics und Hierarchien

Ein Topic ist wie eine Adresse für Nachrichten, z.B.:

```
campus/station-01/environment  
campus/station-02/environment  
campus/+/environment (Wildcard: alle Stationen)
```

Entwerft eine Topic-Hierarchie für ein Szenario mit 3 Stationen (S1, S2, S3) auf dem Campus, einer Auswertungs-App und einer Anzeige-App. Begründet eure Entscheidungen kurz.

Unsere Topic-Hierarchie (mit Begründung):

GEMEINSAM Phase 1 – Spezifikation (ca. 40 min)

💡 Das Ergebnis dieser Phase ist ein verbindlicher Vertrag – beide Gruppen bauen darauf. Unklarheiten hier kosten euch Zeit bei der Integration.

1.1 – Message Schema definieren

Definiert gemeinsam das JSON-Schema für eine Sensor-Nachricht. Für jedes Feld: Name, Datentyp in C, Wertebereich, und was passiert bei einem ungültigen Wert.

Feldname (JSON)	C-Datentyp	Gültiger Wertebereich	Reaktion bei ungültigem Wert

💡 Denkt auch an: Reihenfolge der Felder relevant? Pflichtfelder vs. optionale Felder? Was ist die maximale Länge eines Payloads in Bytes?

1.2 – Funktionssignaturen festlegen

Entwerft für jede der vier Hauptfunktionen eine vollständige Signatur. Schreibt noch keinen C-Code – nur Name, Parameter, Rückgabewert und Randfälle.

Funktion 1 – Datengenerator (wird von Gruppe B implementiert):

Funktion 1	
Name der Funktion	
Aufgabe (ein Satz)	
Parameter (Name + Typ)	
Rückgabewert	
Randfall 1 + Reaktion	
Randfall 2 + Reaktion	

Funktion 2 – JSON-Serializer (wird von Gruppe A implementiert):

Funktion 2	
Name der Funktion	
Aufgabe (ein Satz)	
Parameter (Name + Typ)	
Rückgabewert	
Randfall 1 + Reaktion	
Randfall 2 + Reaktion	

Funktion 3 – Verbindungslogik (wird von Gruppe A implementiert):

Funktion 3	
Name der Funktion	
Aufgabe (ein Satz)	
Parameter (Name + Typ)	
Rückgabewert	
Randfall 1 + Reaktion	
Randfall 2 + Reaktion	

Funktion 4 – Validator (wird von Gruppe B implementiert):

Funktion 4	
Name der Funktion	
Aufgabe (ein Satz)	
Parameter (Name + Typ)	
Rückgabewert	
Randfall 1 + Reaktion	
Randfall 2 + Reaktion	

1.3 – Fehlerszenarien antizipieren

Füllt die Tabelle gemeinsam aus – bevor ihr eine Zeile implementiert habt. Was erwartet ihr? Ihr füllt die letzte Spalte erst in Phase 3 aus.

Was passiert wenn ...	Erwartetes Verhalten	Tatsächliches Verhalten
... der Broker nicht läuft wenn Publisher startet?		
... ein Pflichtfeld im JSON fehlt?		

Was passiert wenn ...	Erwartetes Verhalten	Tatsächliches Verhalten
... ein Wert außerhalb des definierten Bereichs liegt?		
... der Subscriber erst nach 10 Nachrichten startet?		
... zwei Nachrichten mit derselben seq-Nummer ankommen?		
... das Netzwerk während der Übertragung abbricht?		

1.4 – Schätzungen (vor der Implementierung!)

Schätzt jetzt – nach der Spec, aber vor dem ersten Zeilen Code – wie lange jede Aufgabe dauern wird. Ihr vergleicht am Ende.

Aufgabe	Schätzung (min)	Tatsächlich (min)	Differenz
Datengenerator (Gruppe B)			
JSON-Serializer (Gruppe A)			
Verbindungslogik (Gruppe A)			
Validator (Gruppe B)			
Integration Sitzung 1			
Integration Sitzung 2			

 Die Schätzung vor dem Schreiben ist kein Ratespiel – sie ist die Probe, ob ihr die Aufgabe wirklich verstanden habt. Wer nicht schätzen kann, hat die Aufgabe noch nicht verstanden (aber gleich vorweg: das kann man nur halbwegs mit viel Erfahrung, und für große Projekte geht es immer in die Hose...)

IMPLEMENTIERUNG Phase 2 – Sitzung 1 (ca. 90 min)**GRUPPE A – JSON-Serializer**

Eure Aufgabe: eine Funktion `serialize_to_json()`, die einen befüllten Sensor-Struct in einen JSON-String umwandelt – exakt nach der gemeinsamen Spec aus Phase 1.

Ausgangspunkt: `test_publisher.c` zeigt euch, wie Payloads bisher mit `snprintf()` gebaut werden. Eure Funktion ersetzt diesen Inline-Code durch eine saubere, testbare Einheit.

Schritt 1 – Selbst implementieren (kein KI):

1. Schreibt die Funktion nach eurer Spec. Kein copy-paste aus `test_publisher.c`.
2. Testet manuell: Erzeugt einen Struct mit Testwerten, ruft die Funktion auf, prüft den String mit `printf()`.
3. Provoziert mindestens einen Randfall aus eurer Spec – und überprüft, ob die Funktion ihn korrekt behandelt.

Schritt 2 – KI für Randfälle nutzen:

Wählt einen Randfall, den ihr selbst noch nicht sauber gelöst habt. Formuliert einen Prompt – erst schriftlich, dann abschicken:

X Unvollständiger Prompt	✓ Spec-basierter Prompt
Schreib eine C-Funktion die einen Sensor-Struct in JSON umwandelt.	Schreib eine C-Funktion mit dieser Signatur: <pre>int serialize_to_json(const SensorReading *r, char *buf, size_t buf_len);</pre> Regeln: - Gibt 0 zurück bei Erfolg, -1 bei Fehler - Gibt -1 zurück wenn buf NULL oder buf_len < 80 - Feldnamen exakt: seq, station_id, timestamp, temperature_c, humidity_pct - Zahlen mit 1 Dezimalstelle (%.1f) - Kein trailing newline - Keine printf()-Aufrufe in der Funktion

Mein Prompt (vor dem Abschicken aufschreiben):

Gruppe A – Zwischencheck Serializer:

- ☐ Funktion kompiliert ohne Warnings
- ☐ Output entspricht exakt dem vereinbarten JSON-Format (Feldnamen, Reihenfolge, Dezimalstellen)
- ☐ Buffer-Overflow-Fall getestet: was passiert wenn `buf_len` zu klein?
- ☐ NULL-Pointer-Fall getestet
- ☐ Ich kann jede Zeile der Funktion erklären – auch den KI-generierten Teil

GRUPPE B – Datengenerator

Eure Aufgabe: eine Funktion `generate_reading()`, die realistische Sensordaten simuliert – mit langsamem Drift, gelegentlichen Ausreißern und den Fehlerfällen aus eurer Spec.

Ausgangspunkt: `test_publisher.c` erzeugt Zufallswerte mit `rand()`. Eure Funktion ersetzt das durch eine kontrollierte Simulation – jemand anderes muss damit testen können.

Schritt 1 – Selbst implementieren (kein KI):

4. Implementiert den Drift: Temperatur ändert sich pro Schritt um maximal $\pm 0.5^{\circ}\text{C}$ (nicht springen).
5. Baut einen steuerbaren Fehlerfall ein: jede N-te Nachricht soll einen Ausreißer produzieren (N als Parameter).
6. Sequenznummer korrekt hochzählen – auch nach einem simulierten Neustart.

Schritt 2 – KI für den Timestamp nutzen:

Der Timestamp im Format "2024-03-01T08:15:32Z" ist in C nicht trivial. Nutzt KI dafür – aber mit einem Prompt der eure Spec widerspiegelt:

Mein Prompt für die Timestamp-Funktion (vor dem Abschicken aufschreiben):

Gruppe B – Zwischencheck Datengenerator:

- ☐ Temperatur driftet realistisch (kein unkontrolliertes Springen)
- ☐ Ausreißer werden korrekt getriggert (jede N-te Nachricht)
- ☐ Sequenznummer läuft korrekt durch – kein Überlauf bei 20 Nachrichten
- ☐ Timestamp ist ISO 8601-konform und entspricht der Spec
- ☐ Ich kann erklären, was passiert wenn `generate_reading()` 100-mal aufgerufen wird

GEMEINSAM Phase 3 – Integration Sitzung 1 (ca. 30 min)

🕒 Jetzt kommt der Moment der Wahrheit. Gruppe A's Serializer bekommt Daten von Gruppe B's Generator. Wenn es kracht, ist das kein Fehler – es ist Information über eure Spec.

3.1 – Zusammenführen

Verbindet beide Funktionen: Generator liefert einen Struct, Serializer wandelt ihn in JSON um.
Testet zunächst ohne Broker – nur die Funktion-zu-Funktion-Schnittstelle.

Prüfpunkt	Ergebnis	OK?
JSON-Output enthält alle Felder der Spec		✓ / ✗
Feldnamen stimmen exakt überein (Groß-/Kleinschreibung!)		✓ / ✗
Numerische Werte haben korrekte Dezimalstellen		✓ / ✗
Ausreißer-Wert vom Generator wird korrekt serialisiert		✓ / ✗
20 aufeinanderfolgende Nachrichten – keine Abstürze		✓ / ✗
Sequenznummer im JSON stimmt mit Generator-internem Zähler überein		✓ / ✗

3.2 – Wo hat die Spec nicht gereicht?

Jede Abweichung zwischen erwartetem und tatsächlichem Verhalten ist eine Spec-Lücke.
Dokumentiert sie:

Spec-Lücken und Missverständnisse die wir gefunden haben:

GEMEINSAM Reflexion – Ende Sitzung 1 (ca. 20 min)

☞ Diese Reflexion ist Pflicht – nicht optional. Sie ist die Vorbereitung für Sitzung 2. Wer sie überspringt, macht in Sitzung 2 dieselben Fehler noch einmal.

Was hat in der Integration funktioniert – und warum? Was hat nicht funktioniert – und was war die eigentliche Ursache?

Vergleicht eure Zeitschätzungen mit der tatsächlichen Zeit (tragt die Istwerte in die Tabelle auf Seite 3 ein). Wo lagen die größten Abweichungen? Warum?

Wo hat die KI-generierte Funktion nicht exakt eurer Spec entsprochen? Was fehlte in eurem Prompt?

Was werdet ihr in Sitzung 2 an eurem Spezifikationsprozess anders machen?

SITZUNG 2 · 3 Stunden

💡 Bevor ihr anfangt: Lest eure Reflexion von Sitzung 1 nochmal durch. Was habt ihr euch vorgenommen? Haltet ihr das jetzt ein?

IMPLEMENTIERUNG Phase 2 – Sitzung 2 (ca. 90 min)

Neue Schätzungen – vor dem Start

Ihr kennt jetzt eine Aufgabe aus Sitzung 1 gut. Schätzt die beiden neuen Aufgaben – und begründet eure Schätzung.

Aufgabe	Schätzung (min)	Begründung der Schätzung
Verbindungslogik (Gruppe A)		
Validator (Gruppe B)		

GRUPPE A – Verbindungslogik

Eure Aufgabe: eine Funktion `mqtt_connect_and_publish()`, die eine Verbindung zum Broker aufbaut, Nachrichten publiziert und Fehler sauber behandelt – so dass Gruppe B's Subscriber darauf verlassen kann.

Ausgangspunkt: `test_publisher.c` enthält die Verbindungslogik inline in `main()`. Eure Aufgabe: Kapselt das in eine Funktion mit klarem Fehlerpfad. Die Callbacks `on_connect` und `on_publish` dürft ihr adaptieren.

Schritt 1 – Selbst implementieren:

7. Extrahiert die Verbindungslogik aus `main()` in eine eigene Funktion.
8. Behandelt den Fall: Broker nicht erreichbar beim Start. Was soll die Funktion zurückgeben? Was soll `main()` dann tun?
9. Testet mit `mosquitto -d` (Broker läuft) und dann: Broker stoppen, Publisher starten – verhält er sich wie spezifiziert?

Schritt 2 – KI für Reconnect-Logik:

Implementiert eine einfache Retry-Logik (3 Versuche, 2 Sekunden Pause) mit KI-Unterstützung. Prompt erst schriftlich:

Mein Prompt für die Reconnect-Logik:

Gruppe A – Zwischencheck Verbindungslogik:

- ☐ Broker-down-Fall: Funktion gibt definierten Fehlercode zurück (kein Absturz)

- ☐ Retry-Logik: genau 3 Versuche, dann sauberer Abbruch
- ☐ Alle mosquitto_*-Ressourcen werden in jedem Pfad freigegeben (kein Memory Leak)
- ☐ on_connect-Callback setzt internen Status korrekt – keine Race Condition mit dem Hauptthread
- ☐ Verhalten ist für Gruppe B vorhersehbar: sie wissen, wann und in welcher Reihenfolge Nachrichten ankommen

GRUPPE B – Validator

Eure Aufgabe: eine Funktion `validate_message()`, die einen empfangenen JSON-Payload gegen die gemeinsame Spec prüft – vollständig und mit definierten Fehlercodes.

Ausgangspunkt: `test_subscriber.c` gibt jede Nachricht nur aus. Eure Funktion ersetzt das durch eine echte Prüfung – sie ist der Qualitätswächter des Systems.

Schritt 1 – Spec für den Validator schärfen:

10. Definiert Rückgabecodes: 0 = gültig, -1 = fehlendes Feld, -2 = Wert außerhalb Range, -3 = ungültiges Format. Weitere nach Bedarf.
11. Entscheidet: JSON parsen in C selbst (mit `sscanf/strstr`) oder mit einer Mini-Library? Begründet eure Wahl schriftlich.
12. Implementiert die Grundprüfung selbst: alle Pflichtfelder vorhanden?

Schritt 2 – KI für die Wertebereichsprüfung:

Lasst die Wertebereichsprüfung von KI generieren – mit eurer Spec als Prompt. Testet den Output bevor ihr ihn einbaut:

Mein Prompt für die Wertebereichsprüfung:

Unsere Entscheidung zur JSON-Parsing-Strategie und Begründung:

Gruppe B – Zwischencheck Validator:

- ☐ Alle Rückgabecodes sind definiert und dokumentiert
- ☐ Fehlendes Pflichtfeld → korrekt -1 zurückgegeben (getestet mit handgebautem kaputten JSON)
- ☐ Wert außerhalb Range → korrekt -2 zurückgegeben
- ☐ Gültige Nachricht von Gruppe B's Generator → korrekt 0 zurückgegeben
- ☐ Validator baut auf der Spec aus Phase 1 auf – nicht auf Annahmen über Gruppe A's Implementierung

GEMEINSAM Phase 3 – Integration Sitzung 2 (ca. 40 min)

💡 Jetzt laufen alle vier Komponenten zusammen: Generator → Serializer → Broker → Validator. Alles was hier kracht, hat seinen Ursprung in der Spec – nicht im Code.

3.3 – Vollständiger Systemtest

Führt jetzt den kompletten Stack aus: Generator erzeugt Daten → Serializer baut JSON → Publisher sendet an Broker → Subscriber empfängt → Validator prüft.

Prüfpunkt	Ergebnis	OK?
20 gültige Nachrichten ankommen und als gültig erkannt		✓ / ✗
Fehlernachricht (fehlendes Feld) wird als -1 erkannt		✓ / ✗
Fehlernachricht (Wert außerhalb Range) wird als -2 erkannt		✓ / ✗
Ausreißer vom Generator werden korrekt serialisiert UND vom Validator erkannt		✓ / ✗
Broker-Neustart: Publisher reconnected, Subscriber verliert keine Nachrichten		✓ / ✗
Sequenznummern sind lückenlos und aufsteigend		✓ / ✗

3.4 – Fehlerszenarien aus Phase 1.3 verifizieren

Geht zurück zu eurer Tabelle aus Phase 1.3. Füllt jetzt die Spalte "Tatsächliches Verhalten" aus. Was hat euch überrascht?

Die größte Überraschung beim Testen der Fehlerszenarien:

GEMEINSAM Phase 4 – Abschlussreflexion (ca. 30 min)

Beantwortet in eigenen Worten und ganzen Sätzen. Es gibt keine Musterlösung – wir interessieren uns für euer ehrliches Denken.

Zur Spec-Disziplin

Vergleicht eure Spec aus Phase 1 mit dem, was ihr tatsächlich gebaut habt. Wo war die Spec zu vage? Wo zu detailliert? Was hättet ihr anders formuliert?

Es gibt zwei Arten von Spec-Lücken: solche, die ihr beim Schreiben übersehen habt, und solche, die erst durch die Integration sichtbar wurden. Nennt je ein Beispiel aus eurem Labor.

Zur Zusammenarbeit

An welcher Stelle hat die Trennung in Gruppe A und B den größten Lerneffekt gebracht? Und wo hat sie euch behindert?

Stellt euch vor, ihr hättet von Anfang an zusammen alles implementiert. Was hättet ihr nicht gelernt?

Zu KI-Tools

Vergleicht: In Sitzung 1 habt ihr einen Prompt ohne die Erfahrung der Integration geschrieben. In Sitzung 2 mit dieser Erfahrung. War euer Prompt in Sitzung 2 besser? Woran merkt ihr das?

Wo hat KI-generierter Code eure Spec widergespiegelt – und wo hat er Annahmen gemacht, die ihr nicht spezifiziert hattet? Was war die Konsequenz?

Zum großen Bild

In einem Berufsprojekt würde Gruppe A von Gruppe B durch Monate und vielleicht durch verschiedene Zeitzonen getrennt sein. Was ändert sich an den Anforderungen an die Spec – und was bleibt gleich?

Was nehmt ihr aus diesem Labor mit – für die Arduino-Veranstaltung, für euren Umgang mit KI-Tools, und für eure eigene Arbeitsweise?

Anhang: Technisches Setup

 Diese Seite ist eine Referenz – kein Arbeitsauftrag. Wenn etwas nicht funktioniert, ist hier der erste Anlaufpunkt.

Setup in GitHub Codespaces (empfohlen)

Startet euren Codespace, dann im Terminal:

```
sudo apt-get install -y gcc make libmosquitto-dev mosquitto-clients mosquitto
make
mosquitto -d
./test_subscriber      # Terminal 1 - wartet auf Nachrichten
./test_publisher       # Terminal 2 - sendet 5 Nachrichten
```

Bekannte Stolpersteine

Problem	Lösung
Broker läuft nicht	mosquitto -d vergessen. Prüfen: <code>ps aux grep mosquitto</code>
Port 1883 blockiert	Codespaces blockiert ausgehende Ports – nur localhost verwenden, nicht test.mosquitto.org
Subscriber bekommt nichts	Subscriber zuerst starten, dann Publisher. MQTT QoS 1 puffert keine verpassten Nachrichten ohne persistent session.
make: command not found	<code>apt-get install -y make</code> – dann nochmal <code>make</code>
Linker-Fehler: -lmosquitto	In der Makefile-Zeile muss am Ende <code>-lmosquitto</code> stehen

Message Schema (vereinbart in Phase 1 – hier zur Erinnerung)

```
{
  "seq":          1,
  "station_id":   "S1",
  "timestamp":    "2024-03-01T08:15:32Z",
  "temperature_c": 19.3,
  "humidity_pct": 64.2
}
```

 Das ist das Referenz-Schema. Euer tatsächliches Schema aus Phase 1.1 kann abweichen – dann gilt eures, nicht dieses.

Repo-Struktur

Datei	Status	Zweck
-------	--------	-------

test_publisher.c	<i>Vorhanden</i>	Ausgangspunkt für Serializer (A) und Verbindungslogik (A)
test_subscriber.c	<i>Vorhanden</i>	Ausgangspunkt für Validator (B)
Makefile	<i>Vorhanden</i>	Kompiliert beide Programme
CODESPACES_SETUP.md	<i>Vorhanden</i>	Schritt-für-Schritt Setup in Codespaces
publisher.c	<i>Ihr erstellt</i>	Eure finale Version mit Generator + Serializer + Verbindungslogik
subscriber.c	<i>Ihr erstellt</i>	Eure finale Version mit Subscriber + Validator